TruthLens

Multimodal Misinformation Detection System

Product Requirements Document (PRD) — v1.0

Field	Detail
Product Name	TruthLens
Version	1.0 — Hackathon MVP
Date	February 2026
Track	Artificial Intelligence / ML
Author (Backend)	Backend Lead
Status	Active Development

1. Executive Summary

TruthLens is an AI-powered, multimodal misinformation detection platform that analyzes text, images, audio, video, and URLs to produce structured credibility reports with transparent reasoning, source citations, and ethical AI safeguards.

> *Core Promise: Every piece of content gets a credibility score, a clear verdict, flagged claims with sources, and an honest explanation of the AI's confidence — never a black-box answer.*

Unlike existing tools that operate on a single modality or return opaque verdicts, TruthLens unifies the entire detection pipeline under one API, making it practical for journalists, researchers, general users, and enterprise platforms.

2. Problem Statement

2.1 The Misinformation Crisis

Misinformation spreads 6× faster than factual content online (MIT Media Lab, 2018). The landscape has worsened significantly with the rise of:

- Generative AI that can fabricate realistic text, images, audio, and video at scale

- Deepfake technology accessible to non-experts via consumer apps
- Social media amplification that rewards engagement over accuracy
- Cross-modal attacks — false narratives reinforced by manipulated visuals

2.2 Gaps in Existing Solutions

Existing Tool	Limitation
Snopes / PolitiFact	Manual, slow, text-only, English-centric
Google Fact Check	Requires prior publication of fact-check, no real-time
Deepware Scanner	Video/deepfake only, no text or audio
Hive Moderation	Paid, image-only, no reasoning or source citations
Generic LLM chatbots	No grounding, hallucinate, no explainability layer

Gap: No unified, explainable, multimodal, free-tier-accessible tool exists for real-time misinformation detection across all content types.

3. Proposed Solution

3.1 System Overview

TruthLens ingests any content type and normalizes it into a unified text + metadata corpus before running it through a multi-stage analysis engine. Every input type — no matter how complex — converges to the same high-quality analysis pipeline.

3.2 Architecture Pipeline

Stage	Description	Technology
Input Layer	Accept text, URL, image, audio, video	FastAPI multipart + JSON
Normalization	Convert all inputs to text corpus + metadata	BeautifulSoup, MoviePy, Whisper, Vision LLM
Frame Extraction	Sample key frames from video (1/5s, scene-change filter)	OpenCV / MoviePy
Claim Extraction	Pull verifiable factual claims from corpus	Groq LLaMA-3.3-70b
Fact Verification	Cross-reference claims against fact-check databases	Google Fact Check API
Frame Analysis	Analyze sampled frames for AI generation & manipulation	OpenRouter Vision LLM (LLaVA / Qwen-VL)

Linguistic Analysis	Detect sensationalism, emotional manipulation, logical fallacies	Groq LLaMA-3.3-70b
Aggregation Engine	Weighted scoring across all module outputs	Python — custom logic
Report Generation	Produce structured JSON + human-readable report	Groq LLM + template engine

3.3 Video Frame Extraction Strategy

Video analysis is one of TruthLens's key differentiators. Rather than processing every frame (expensive and slow), TruthLens uses intelligent sampling:

- Extract 1 frame per 5 seconds → 12 frames per minute of video
- Apply scene-change detection (pixel diff threshold) to remove near-duplicate frames
- Send 8–12 key frames maximum to Vision LLM for analysis
- Cap input video at 2 minutes; extract audio track separately for Whisper transcription

Result: Full video analysis in under 30 seconds for a 1-minute clip, with both visual and spoken content analyzed.

4. Output Specification

4.1 Unified Report Schema

Every analysis — regardless of input type — returns the following structured JSON:

```
{
  "id": "uuid-v4",
  "verdict": "LIKELY MISLEADING",
  "credibility_score": 31,
  "confidence": 0.82,
  "summary": "Natural language explanation of verdict...",
  "claims_analyzed": [{ "claim": "...", "verdict": "FALSE",
    "source": "WHO, CDC", "explanation": "..." }],
  "red_flags": ["sensationalist_language", "no_sources"],
  "frame_analysis": { "frames_analyzed": 10,
    "manipulation_detected": true, "ai_generated_score": 0.74 },
  "ethical_notes": { "bias_warning": "...", "privacy": "..." },
  "input_type": "video",
  "processing_path": "video → frames + audio → Whisper → LLM",
  "timestamp": "2026-02-27T10:00:00Z"
}
```

4.2 Verdict Scale

Score Range	Verdict	Meaning
80–100	CREDIBLE	High confidence, no significant red flags
60–79	MOSTLY CREDIBLE	Minor issues; verify key claims
40–59	UNCERTAIN	Mixed signals; independent verification needed
20–39	LIKELY MISLEADING	Multiple red flags or false claims detected
0–19	HIGHLY DECEPTIVE	Strong evidence of fabrication or manipulation

5. API Design

5.1 Endpoints

Method	Endpoint	Input	Description
POST	/analyze/text	JSON: {content}	Analyze raw text
POST	/analyze/url	JSON: {url}	Scrape + analyze URL
POST	/analyze/image	multipart: file	Analyze image for AI manipulation
POST	/analyze/audio	multipart: file	Transcribe + analyze audio
POST	/analyze/video	multipart: file	Extract frames + audio, full analysis
GET	/report/{id}	Path param	Retrieve stored report by ID
GET	/health	—	Service health check
GET	/docs	—	Auto-generated Swagger UI (FastAPI)

5.2 Response Codes

Code	Meaning
200	Success — analysis complete
202	Accepted — processing async (large video)
400	Bad request — invalid input or file type
413	Payload too large — file exceeds size limit
429	Rate limit exceeded
503	Upstream AI service unavailable — partial results returned

6. Technology Stack

Layer	Technology	Purpose	Cost
API Framework	FastAPI (Python)	Async REST API + auto Swagger docs	Free
LLM — Reasoning	Groq (LLaMA-3.3-70b)	Claim extraction, analysis, report gen	Free tier
LLM — Vision	OpenRouter (LLaVA / Qwen-VL)	Image & frame analysis	Free credits
Audio → Text	OpenAI Whisper (local)	Audio & video transcription	Free / local
Fact Check	Google Fact Check Tools API	Cross-reference claims	Free
URL Scraping	httpx + BeautifulSoup	Extract article text from URLs	Free
Video Processing	MoviePy + OpenCV	Frame extraction, scene detection	Free
Caching	Redis (or in-memory dict)	Cache repeat URL analyses	Free
Frontend	React + Tailwind + Framer Motion	UI (teammate)	Free
Hosting	Render / Railway	Deployment + live demo URL	Free tier

7. Feature Specification

7.1 Core Features (MVP — Must Ship)

Feature	Priority	Input Types	Notes
Text analysis + credibility score	P0	Text, URL	Foundation of the system
Claim extraction + fact-check	P0	Text, URL, Audio	Google Fact Check API
URL scraping pipeline	P0	URL	httpx + BeautifulSoup
Audio transcription + analysis	P0	Audio, Video	Whisper local model
Image: vision LLM analysis	P0	Image, Video frames	AI-gen + manipulation flags
Video frame extraction	P0	Video	Scene-aware sampling
Unified report generator	P0	All	Structured JSON + summary
Explainability layer	P0	All	Never a black box
Ethical AI notes in output	P0	All	Bias, confidence warnings
FastAPI auto-docs (Swagger)	P0	—	Judges see live API docs

7.2 Enhanced Features (Ship if Time Allows)

Feature	Priority	Est. Time
Redis caching for repeat URLs	P1	30 min
Rate limiting middleware	P1	20 min
Async processing + job queue for large video	P1	45 min
Browser Extension (lite)	P2	60–90 min
Demo video	P2	30 min
Feedback endpoint (user reports wrong verdict)	P3	45 min

8. Reliability & Scalability Design

8.1 Reliability Patterns

- **Graceful Degradation — if Vision LLM is unavailable, return text-only analysis with a note. Never fail silently.**
  - Each module returns partial results independently
  - Aggregation engine handles missing module outputs gracefully
- **Fallback Chain — Groq → OpenRouter → cached response → error with explanation**
- **Input Validation — file type whitelist, size limits, timeout guards**
- **Confidence Thresholds — system outputs 'UNCERTAIN' rather than wrong confident answers when evidence is weak**

8.2 Scalability Design

- Stateless API — all state in Redis or DB, allowing horizontal scaling behind load balancer
- Async FastAPI — all I/O-bound operations (LLM calls, scraping) are non-blocking
- Response Caching — identical URL/text input hits cache, not AI APIs (saves cost + latency)
- Video Size Cap — 50MB / 2 minutes max; frame extraction is CPU-bound, documented in API
- Modular Architecture — each analysis module is independently deployable as a microservice in production

Scalability Note: The architecture is designed so any module can be swapped for a more powerful (or cheaper) model without changing the API contract. This is the foundation for production scaling.

9. Ethical AI & Privacy

9.1 Core Ethical Principles

Principle	Implementation
No Binary Verdicts	All outputs are probabilistic with confidence scores, not absolute true/false
Explainability First	Every verdict includes reasoning and sources — users understand why
Bias Acknowledgment	Politically/socially sensitive topics include explicit bias warnings in output
Human in the Loop	System aids human judgment, never replaces it — stated in all reports
False Positive Handling	System uses 'UNCERTAIN' band for ambiguous content, not a wrong verdict
Feedback Mechanism	Users can flag incorrect verdicts (P3 feature) to improve the model

9.2 Privacy Safeguards

- No user content stored beyond the analysis session by default
- No PII collected — no login required for demo
- Uploaded files deleted from server after processing (configurable retention)
- All API calls to external services (Groq, OpenRouter) use data minimization — only the text corpus is sent, not raw files
- Privacy statement included in every API response under ethical_notes field

10. Testing & Validation Plan

10.1 Test Cases (Run Before Demo)

#	Input	Expected Verdict	Pass Condition
T1	Known fake news article URL (e.g. from MediaBias/FactCheck)	LIKELY MISLEADING	Score < 40

T2	Legitimate Reuters/AP news article URL	CREDIBLE	Score > 70
T3	Plain text: 'Vaccines cause autism'	HIGHLY DECEPTIVE	Claim flagged, sources cited
T4	AI-generated image (from known AI art sites)	Manipulation detected	frame_analysis.ai_generated_score > 0.6
T5	Real photograph from news archive	No manipulation	frame_analysis.ai_generated_score < 0.4
T6	Audio clip of a real speech	Transcription accurate	Whisper output matches known text
T7	Short video with false narration	MISLEADING	Audio + frame flags combined
T8	Empty input / unsupported file type	400 Bad Request	Graceful error, no crash
T9	Same URL submitted twice	Same result, faster	Cache hit logged

10.2 Evaluation Metrics

Metric	Target	How to Measure
API response time (text)	< 5 seconds	Postman / curl timer
API response time (video)	< 45 seconds	Stopwatch on 60s clip
Fact-check accuracy	> 80% on known cases	Run T1–T7 test suite
Crash rate	0% on valid inputs	Run all test cases
Frame extraction (1 min video)	8–12 key frames	Log output

11. Hackathon Execution Timeline

Hour Block	Tasks	Owner	Deliverable
Hour 1	Repo setup, FastAPI scaffold, env vars, project structure	Backend	Running hello-world API
Hours 2–3	Text analysis module — Groq LLM + prompt engineering	Backend	POST /analyze/text working
Hours 3–4	URL scraper + Google Fact Check API integration	Backend	POST /analyze/url working
Hours 4–5	Whisper audio module + OpenRouter image/vision module	Backend	POST /analyze/audio + /image

Hours 5–6	Video pipeline — MoviePy frame extraction + scene detection	Backend	POST /analyze/video
Hours 6–7	Aggregation engine + unified report generator	Backend	All endpoints return full report
Hours 7–8	Error handling, fallbacks, caching, input validation	Backend	Reliable, no crashes
Hours 8–9	End-to-end testing (T1–T9 test suite)	Backend	All tests passing
Hour 9–10	Deploy to Render/Railway — live public URL	Backend	Demo-ready
Hours 10–11	PPT (6 slides) + API documentation	Both	Submission-ready
Hours 11–12	Buffer: Browser extension OR polish OR demo video	Backend	Bonus points

12. Browser Extension — Phase 2 Scope

Status: Included in roadmap. Attempt in final hour only if API is fully deployed and stable.

The browser extension is a thin wrapper around the TruthLens API. You already have the hard part — the backend. The extension is 3 files and ~100 lines of code total.

12.1 Extension Architecture

- manifest.json — extension config, permissions (activeTab, contextMenus)
- popup.html — minimal UI: TruthLens logo, analyze button, result display
- popup.js — grabs current tab URL, calls POST /analyze/url, renders score + verdict

12.2 User Flow

User visits a suspicious news article → clicks TruthLens icon in toolbar → popup shows credibility score + verdict + top red flags → link to full report. That is the entire product.

Note: No browser extension experience needed. The code will be generated for you once the API is deployed. You only need to replace one line (the API_BASE_URL) with your live Render URL.

13. Submission Checklist

Item	Status	Notes
Idea Presentation PPT (4–6 slides)	Required	Architecture, demo, impact, ethics
Working Prototype / Demo Link	Required	Live Render/Railway URL
Brief Project Documentation	Required	This PRD serves as documentation
Team Information	Required	Team Name / Leader / College
Demo Video	Bonus	Record screen walkthrough — upload to Drive link
Browser Extension	Bonus	Attempt in final hour
Future Roadmap slide	Optional	Phase 2: multilingual, enterprise API, mobile

14. Impact & Future Roadmap

14.1 Immediate Impact

- Journalists can verify breaking news content in seconds before publishing
- Researchers get a free, open API for misinformation dataset creation
- General users get a browser extension for day-to-day content verification
- Platform teams can integrate the API to auto-flag content before it goes viral

14.2 Future Roadmap (Post-Hackathon)

Phase	Feature	Timeline
Phase 2	Multilingual support (Hindi, Spanish, Arabic)	Month 1–2
Phase 2	Dedicated deepfake detection model (fine-tuned)	Month 2–3
Phase 3	Enterprise API with SLA + usage dashboard	Month 3–4
Phase 3	Mobile app (iOS + Android)	Month 4–6
Phase 4	Real-time social media stream monitoring	Month 6+
Phase 4	Federated learning for privacy-preserving model improvement	Month 9+

Final Note: A well-executed working solution with practical relevance is more valuable than a complex but incomplete concept. TruthLens is designed to be exactly that — practical, explainable, and impressive.